⚡ BURSTY TRAFFIC

Managing Bursty Traffic in Event-Driven Systems

Group 3 · High-Throughput

강성욱 · 심규상 · 이윤서 · 하헌철

Computer Networks · Intermediate Proposal

AGENDA

What we'll cover today

01 **Motivation**
Why bursty traffic matters in modern services

02 **Problem Definition**
The pain of provisioning for the peak

03 **Live Demo**
Watching a 3-server system get crushed

04 **Solution A — Cloud Bursting**
Scale capacity on demand

05 **Solution B — Virtual Waiting Room**
Manage demand with a queue

06 **Synergy & Case Studies**
Real-world combinations that work

Not all high-throughput traffic looks the same

CONSTANT THROUGHPUT

Streaming services

YouTube, Netflix, Twitch — high but stable load throughout the day. Capacity planning is predictable.

low variance · stable

BURSTY TRAFFIC

Ticketing, live-commerce

Concert tickets, course registration, K-pop live drops — idle for hours, then 10–100× spike within seconds.

high variance · unpredictable peaks

Bursty workloads are everywhere now

Ticketing

Concerts, sports events. Sells out in seconds.

Course Registration

University semesters open — every student at once.

Live-commerce

Limited-stock flash drops on home-shopping streams.

Game Launches

Day-one logins overwhelm authentication servers.

Designing for the peak is now a core requirement — not an optional optimization.

The provisioning dilemma

If demand spikes 100× for ten minutes per month — how many servers do you buy?

Provision for the PEAK

~95%

of capacity sits idle

- Servers idle most of the time
- Operating cost is massive
- Still risky if peak underestimated

Provision for the AVERAGE

~90%

of users get dropped at peak

- Cheap during normal hours
- Service collapses during bursts
- Brand damage, lost revenue

Let's watch a 3-server system get crushed

DEMO SETUP

Fixed servers	**3 on-prem (S1, S2, S3)**
Capacity	**~150 req / second**
Normal traffic	**~25–80 req / second**
Burst traffic	**~1,000–1,200 req / second**
Headroom	**8× over capacity**

→ Click '🎫 티켓 오픈!' to trigger the burst

Without protection: the system collapses

1,200	150	8×	145
Peak incoming RPS	Server capacity	Demand over capacity	Requests dropped

WHAT HAPPENED

The RPS curve doesn't just fall — it **drops off a cliff**. This isn't traffic going away. **It's the server dying.** Threads exhaust, response times explode, and the service becomes effectively unreachable.

Failure mode: *the service doesn't slow down —* ***it dies.***

Two complementary strategies

There are two ways to absorb a burst — and the best systems use both.

STRATEGY A

Hybrid Cloud Bursting

Expand the supply side

When on-prem capacity is exhausted, **automatically rent cloud servers** to handle the overflow. Release them when the burst is over.

- 💰 Pay only during the spike
- ⚡ Spin-up takes seconds to minutes
- 🔗 Requires hybrid networking design

STRATEGY B

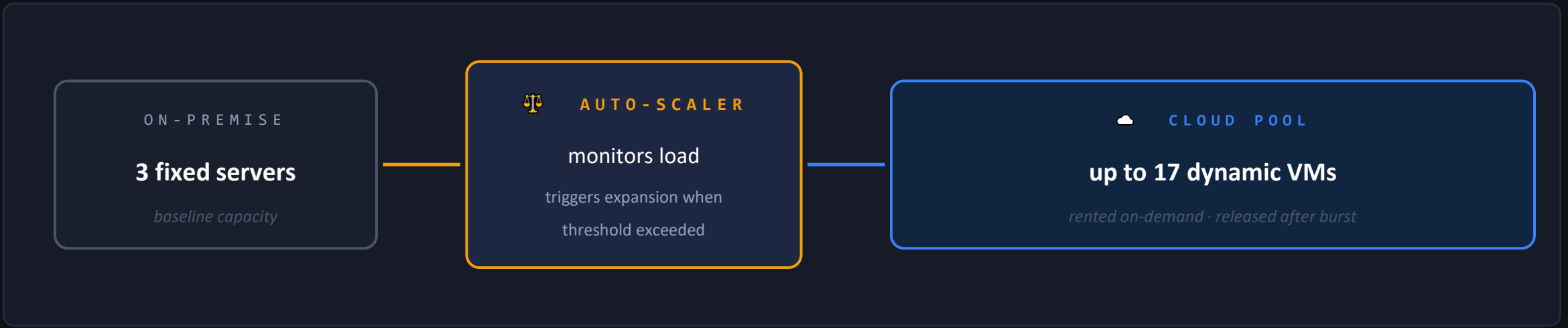
Virtual Waiting Room

Smooth the demand side

Put excess users in a **fair, ordered queue** instead of dropping them. Admit them at a rate the backend can handle.

- 🛡️ Backend stays protected
- ⏱️ No need to scale infrastructure
- 😣 Users wait — UX trade-off

How Hybrid Cloud Bursting works



💰 Cost-efficient

Pay only for cloud capacity during the spike. Idle hours cost nothing.

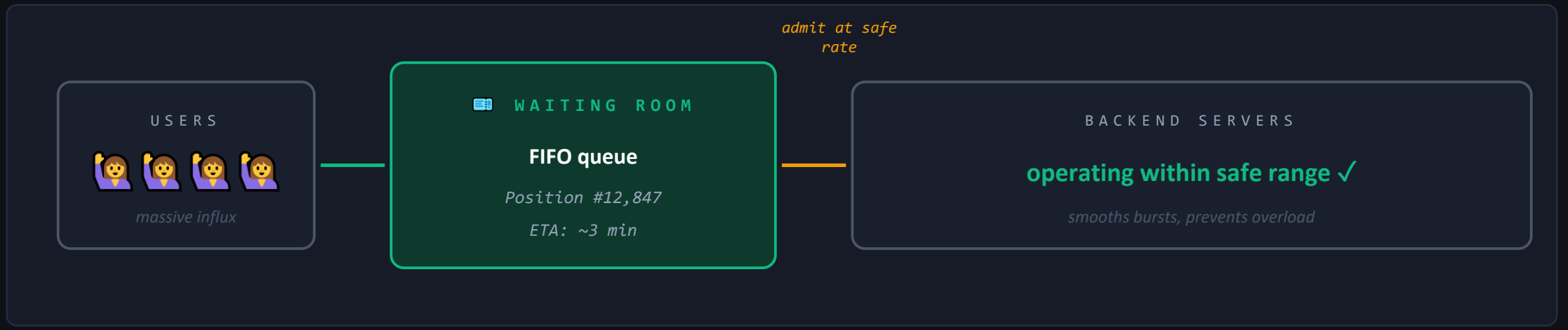
📈 Highly elastic

Scale from 3 to 20 nodes in minutes. No upfront hardware investment.

🔗 Network-dependent

Needs hybrid networking (VPN, Direct Connect) and synced data — non-trivial.

How a Virtual Waiting Room works



🛡️ Protects the backend

Throttles inflow so servers stay in their safe range. Drops drop dramatically.

⚖️ Fair & transparent

FIFO ordering with visible queue position. Reduces frustration.

⏳ UX cost is real

Users wait. Mitigated with progress info and accurate ETAs.

Three scenarios, side by side

BURST · NO PROTECTION

Service dies

DROPPED

145

- 3 on-prem, both toggles OFF
- Capacity: 150 RPS
- Incoming: ~1,200 RPS
- Servers crash, RPS cliff drops

BURST · BOTH ON

Survives, but not perfect

DROPPED

~30

- Cloud servers spin up
- Queue absorbs most traffic
- Some drops still happen
- Service stays alive ✓

NORMAL · WAITING ROOM ON

Zero drops

DROPPED

0

- Normal traffic ~30–80 RPS
- Capacity has plenty of headroom
- Queue smooths micro-bursts
- Steady-state perfection

*Defense isn't perfect under extreme bursts — **but it's massive.** Under normal load, it's perfect.*

Why drops still happen under extreme bursts

When traffic is 8× capacity, no architecture can catch every request. Here's why:

01

Cloud cold start

Detecting overload → spinning up VMs → ready to accept traffic takes seconds to a minute.

→ *During that startup window, incoming traffic outpaces capacity growth. Gap = drops.*

02

Queue size limit

The waiting room queue is large, but not infinite. An unbounded queue would just push failure to memory exhaustion.

→ *When inflow >> drain rate for too long, the queue fills. Then new requests get dropped at the door.*

03

Network-layer limits

Before requests even reach the app, the load balancer and OS have their own caps — TCP backlog, file descriptors, ephemeral ports.

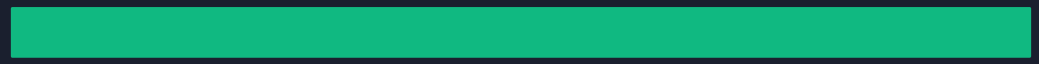
→ *At thousands of RPS instantaneously, some requests get dropped at the kernel level before our code sees them.*

Why the Waiting Room hits 0 drops in normal traffic

THE NUMBERS

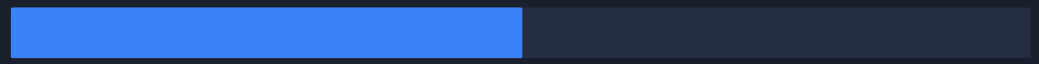
Capacity

150 RPS



Normal demand

~30–80 RPS



HEADROOM

~50%

spare capacity at all times

THE LOGIC

① Demand stays well under capacity

Average ~50 RPS vs 150 RPS capacity. Plenty of room.

② But traffic isn't smooth

Micro-bursts happen: TCP retransmits, batch jobs, users clicking at the same second.

③ Waiting Room absorbs the bumps

Micro-spikes get queued briefly, then drained at a steady rate. Users don't even notice.

Think of it as a shock absorber for traffic — soaks up bumps before they become drops.

Which strategy, and when?

BURST SIZE

HIGH ↑

Just scale vertically

Burst is rare and small. A bigger fixed server is enough.

☁ Cloud Bursting

Predictable timing (e.g. ticket-open events). Pre-warm cloud nodes ahead of the burst.

Waiting Room

Demand is huge but unpredictable. Throttle the inflow to protect the backend.

☁ + Both

Massive, predictable bursts (ticketing). Scale supply AND smooth demand together.

↓ LOW

← LOW

HIGH →

BURST PREDICTABILITY

Real-world deployments

CLOUD BURSTING

AWS × Rackspace

Early hybrid bursting case: workloads ran on Rackspace, burst capacity rented from AWS during traffic spikes. Demonstrated the multi-cloud bursting pattern.

MULTI-CLOUD NETWORKING

AWS × Google Cloud

Recent (2025) collaboration to simplify multi-cloud networking — making hybrid/burst architectures easier to build, with unified routing and identity.

WAITING ROOM

Interpark · Yes24 (Korea)

Ticketing platforms with infamous spikes. Use queue-based admission with position numbers and ETA to prevent backend collapse during high-demand sales.

BURST CREDITS

AWS Burstable Instances

T-series EC2 instances accumulate CPU credits during idle, spend them during bursts. Cloud-native primitive for small, predictable bursts at the VM level.

KEY TAKEAWAYS

Design for the peak — with money or with time.

01

Bursty ≠ rare

Ticketing, live-commerce, registration — bursty is the new normal.

02

Over-provisioning loses

Static capacity wastes 95% in idle hours and still fails at peak.

03

Defense isn't perfect

Extreme bursts still drop some requests — but turn total outages into minor inconvenience. Normal traffic stays at zero drops.

Thank you.

Questions & Discussion — Group 3 · High-Throughput